# Robot Agentic

## The code explained

Software anomaly detection on URSim: every class, its fields and their types, why it exists, a real example, and who uses it. Plus the libraries, the size of the code and what is still missing.

Branch `refactor/software-detection`

State of the code on 1 November 2026

Written to be read on a phone or a tablet. Words in `typewriter` are names in the code. **Every example output in green boxes was produced by running the code on what URSim really wrote during the fault captures of 28 October 2026**, not invented. Chapter 5 explains the Python notions the rest relies on; the glossary at the end explains the robot words.

# Contents

**1 The project in one page** **3**

**2 The big picture** **4**

**3 How a run goes, step by step** **5**

3.1 `robot-agentic check "the arm struggles"` . . . . . 5

3.2 `robot-agentic loop` . . . . . 6

3.3 `robot-agentic health -api` . . . . . 6

**4 Languages, libraries and tools, and why** **6**

4.1 Inside the program . . . . . 6

4.2 Python's own library (no install) . . . . . 7

4.3 For development . . . . . 7

4.4 Outside the program . . . . . 7

4.5 What was *not* used, on purpose . . . . . 7

**5 The Python building blocks used in this code** **7**

5.1 Class and object . . . . . 7

5.2 Type annotations . . . . . 8

5.3 Dataclass: a class that only holds data . . . . . 8

5.4 Enum: a closed list of allowed values . . . . . 9

5.5 Abstract class: a contract . . . . . 9

5.6 Protocol: a contract without inheritance . . . . . 9

5.7 TypedDict: a dictionary with known keys . . . . . 9

5.8 Generator: a function that produces items one by one . . . . . 9

5.9 Threads, queues, locks, events . . . . . 10

5.10 deque: a list that forgets its oldest items . . . . . 10

5.11 numpy array and NaN . . . . . 10

**6 The shared objects: `core/events.py`** **10**

**7 The contract: `core/interfaces.py` 14**

**8 The memory: `core/recorder.py` 15**

**9 The rules without AI: `core/checks.py` 18**

**10 Safety: `core/safety.py` 20**

**11 The URSim connector: `connectors/ursim.py` 21**

11.1 The robot's doors (network ports) . . . . . 21

11.2 The functions . . . . . 21

**12 The agent: `src/robot_agentic/agent/` 24**

**13 Around the agent 28**

**14 Outside the package: tools, benchmarks, tests 30**

**15 How it was developed 31**

15.1 The code principles (CONTRIBUTING.md) . . . . . 31

15.2 The history of this branch . . . . . 31

15.3 Bugs found only by testing on the real simulator . . . . . 32

**16 How big is it? 32**

**17 What is missing 33**

17.1 1. The detection rules on the signals . . . . . 33

17.2 2. The expected current in the live stream . . . . . 33

17.3 3. A "stream health" rule (proposed rule g) . . . . . 33

17.4 4. Benchmarks: never run, incomplete . . . . . 33

17.5 5. Smaller gaps, some found while writing this guide . . . . . 34

17.6 6. Planned for later . . . . . 34

**18 Glossary 34**

# 1 The project in one page

**The goal.** When an industrial robot breaks down, a technician looks at what the robot reports, makes hypotheses, tests them, and finds the cause. Robot Agentic does the same thing with an AI agent (Claude). It reads the robot's **four data sources**:

1. the **signals**: joint positions, speeds, currents, torques, temperatures, 500 times per second;
2. the **logs**: the text lines the robot's programs write;
3. the **fault codes**: the official error numbers of the controller (e.g. `C209A0`);
4. the **documentation**: the official manual pages.

It then runs small, slow, operator-approved tests on the robot, and hands in a report: *the root cause, the evidence, the repair.*

**Two independent approaches.** You defined two ways of detecting anomalies, that never mix in one investigation:

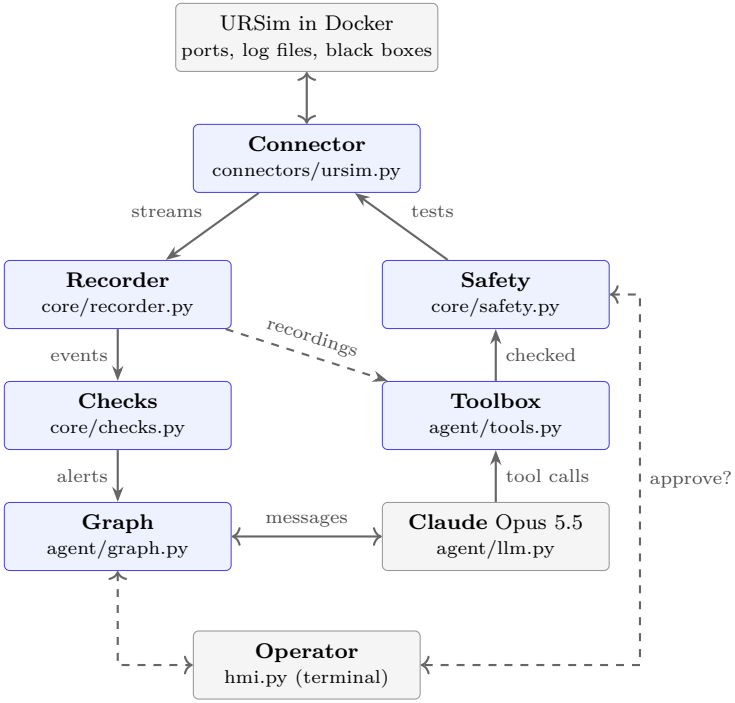
- **software anomaly detection**: uses only what the robot's software reports (the four sources above). *This is the only one in the code today.*
- **physical anomaly detection**: uses our own sensors (microphone, accelerometer, current clamp) on the machine, even a closed one. Planned for later.

**The test bench.** We use **URSim**, the official Universal Robots simulator: the real controller software, running in a Docker container on the laptop. To the code, URSim looks exactly like a real UR3e robot: same network ports, same binary formats, same logs.

**Three commands.**

- `robot-agentic check`: a spot check, on demand, or a question ("the arm struggles when lifting").
- `robot-agentic loop`: continuous monitoring. Simple rules (no AI, no cost) watch the logs and fault codes; when something looks wrong, the operator is asked whether to start an investigation.
- `robot-agentic health`: checks that everything is connected.

# 2 The big picture




1. **The connector** is the only part that knows URSim. It turns raw bytes and text into a few simple Python objects (`Sample`, `LogLine`, `FaultCode`, chapter 6).
2. **The core** (no AI, no robot brand): the **recorder** keeps the last two minutes of every stream in memory; the **checks** raise alerts; the **safety** layer refuses or asks before any movement.
3. **The agent**: the **graph** drives an investigation, the **toolbox** runs what Claude asks for, **context** turns recordings into plots and tables, **llm** talks to Claude.
4. **Around it**: the command line, the terminal interface, the settings, the documentation search; and outside the package, the fault tools, the benchmarks, the tests.

**Why this cut?** Plugging in a new machine (the RoArm-M2-S, a car through OBD-II…) means writing one new connector file. Nothing else changes, because the rest of the code only knows the simple objects and the `Connector` contract.

### The life of one fault, from URSim to the operator

To see how the classes fit, here is what really happened in the code when we sent `protective_stop()` to URSim on 28 October 2026 at 23:32:12. Each class named here is explained in its own section later.

1. URSim's controller writes in its log file: `2026-10-28_23:32:12:07913 00:12:07:710 WARNING - Protective stop. Error code: C209A0`
2. `URSimConnector.logs()` reads that line and `parse_log_line` turns it into a `LogLine` object: level `WARNING`, source `controller`, text `Protective stop. Error code: C209A0`.
3. 24 ms after the command, the Primary port sends a binary packet; `parse_primary` turns it into a `FaultCode`: code `C209A0`, level `WARNING`.
4. In the Real-Time stream, the `Sample`s show `safety_mode` going from 1 to 3 at +0.027 s, and `speed_scaling` dropping to 0.
5. The `Recorder` stores all of it: the `LogLine` and `FaultCode` in its event lists, the `Sample` values in its numpy ring buffer.
6. The `EventWatcher` receives the `FaultCode` and the `LogLine` and produces two `Alert`s: *warning: fault code C209A0*, and *critical: log: Protective stop. Error code: C209A0* (the text contains "protective stop").
7. In loop mode, the graph shows the alert and asks the operator whether to investigate.
8. If yes, the `Toolbox` gives Claude the logs, fault codes and black box; Claude may ask for a test, which the `SafetyPolicy` checks and the operator approves; Claude finally calls `submit_report`.

# 3 How a run goes, step by step

## 3.1 `robot-agentic check "the arm struggles"`

1. `__main__.py` reads the `Settings`, creates the `URSimConnector`, starts the `Recorder` (three background threads), and builds the `Toolbox`.
2. The graph starts at the `investigate` step. It sends Claude a first message with the request, the robot state, the fault codes and the log warnings of the last two minutes.
3. Claude thinks, then calls tools: e.g. `search_docs("C209")`, then `joint_test(joint=1, delta_deg=3)`.
4. For a test, the `SafetyPolicy` checks the bounds, asks the operator, and asks the PIN once.

5. The connector sends a tiny URScript program; the `Recorder` cuts out everything that happened from 1 s before to 1.5 s after the test into a `Recording`.
6. Claude receives a text summary, a plot, and the full values as a table.
7. This repeats (25 turns maximum) until Claude calls `submit_report`. The report is saved as JSON in `data/reports/` and shown in the terminal, with the cost.

### 3.2 `robot-agentic loop`

1. The graph starts at `watch`: it reads every new `LogLine` and `FaultCode`. No Claude call, no cost.
2. The `EventWatcher` decides whether an event deserves an `Alert`.
3. The operator is asked: "*Anomaly: fault code C209A0. Start a root-cause analysis?*"
4. If yes: the same investigation as above, then monitoring resumes.

### 3.3 `robot-agentic health -api`

One line per check: Dashboard reachable and robot RUNNING, Real-Time stream with its rate and channel count, log stream, fault stream, documents found, and (with `-api`) access to the Claude model through a free request.

## 4 Languages, libraries and tools, and why

### 4.1 Inside the program

| | |
|---|---|
| **Python 3.11+** | The language of robotics and AI tooling; every library below exists for it; it runs on the Raspberry Pi. |
| **anthropic (2.3)** | Official Claude SDK: retries, authentication, adaptive thinking, effort, prompt caching, server fallback. |
| **LangGraph (1.2)** | Describes the investigation as a small graph of steps. You chose to keep it: the flow is visible, and adding a step later is one node. |
| **numpy (2.5)** | The ring buffer of signals, resampling, cut detection. |
| **matplotlib (3.11)** | Draws the plot sent to Claude after each test. |
| **pypdf (6.21)** | Reads PDF manuals, if PDFs are added to `manuals/`. |
| **python-dotenv** | Reads the `.env` file (API key, robot address, PIN). |
| **PyYAML** | Reads the scenarios and the catalogue sources (later the detection thresholds). |

### 4.2 Python's own library (no install)

`socket` and `struct` (talk to the robot's ports and decode its binary packets), `subprocess` (run `docker` commands), `threading` and `queue` (the reader threads), `dataclasses`, `enum`, `abc`, `typing` (the building blocks of chapter 5), `collections.deque`, `zipfile` (black boxes), `gzip`/`json` (recordings), `logging`, `argparse`.

### 4.3 For development

**pytest** (24 tests in seconds, no robot, no network, no cost), **ruff** (style, and the two rules a tool can check: type annotations and docstrings everywhere), **setuptools** (creates the `robot-agentic` command).

### 4.4 Outside the program

**Docker** (runs URSim, and provokes faults: pause, CPU limit, network disconnect), an **Alpine helper image** (`tc` and `iptables` for network faults), **udisks** (URSim's own 512 MB disk), **LaTeX** (the catalogue and this guide).

### 4.5 What was *not* used, on purpose

- **pydantic** (validated models): too slow at 500 samples/s, and each connector already parses strictly. Plain dataclasses instead.
- **pandas**: heavy for a Raspberry Pi; numpy is enough.
- **A vector database / embeddings**: fault codes and part names are exact words, which keyword ranking (BM25) handles better, with zero dependency and zero cost.
- **LangChain** around Claude: the loop is 60 lines and fully visible.
- **The UR RTDE library**: we read the Real-Time port directly with `struct`.

## 5 The Python building blocks used in this code

The rest of the guide uses these notions constantly. Each one is shown with an example taken from our own code.

### 5.1 Class and object

A **class** is a mould; an **object** (or *instance*) is one thing made with it. `FaultCode` is a class: it says "a fault code has a time, a code, a message and a level". The fault URSim sent at 23:32:12 is one object made from it:

```
fault = FaultCode(code="C209A0", message="[UNDEFINED]", level=Level.WARNING)
fault.code        # -> "C209A0"
```

A class can also hold **methods**: functions that work on the object's data, written `object.method()`, e.g. `recorder.window(t0, t1)`.

## 5.2 Type annotations

Every field and every function says which type of value it holds or returns. Python does not enforce them when the program runs, but `ruff` and editors use them to catch mistakes, and they tell *you* what to expect. The types you will meet:

| Type | Means | Example |
|---|---|---|
| `float` | a decimal number | `0.6727` |
| `int` | a whole number | `5` |
| `str` | text | `"C209A0"` |
| `bool` | true or false | `True` |
| `list[str]` | an ordered list of texts | `["a", "b"]` |
| `dict[str, float]` | a table from names to numbers | `{"robot_mode": 7.0}` |
| `tuple[bool, str]` | a fixed pair | `(True, "dashboard: ...")` |
| `str \| None` | a text, or nothing at all | `None` |
| `Path` | a file or folder path | `Path("data/reports")` |
| `np.ndarray` | a numpy array of numbers | `array([0.1, 0.2])` |
| `Callable[[str], bool]` | a function taking a text, returning yes/no | `hmi.confirm` |
| `Iterator[Sample]` | an endless supply of samples | `connector.signals()` |
| `Any` | anything (only for JSON from Claude) | |

## 5.3 Dataclass: a class that only holds data

Most of our classes are **dataclasses**: you list the fields with their types, and Python writes the boring code for you (creating the object, printing it, comparing two of them).

```
@dataclass(kw_only=True)
class LogLine:
    t: float = field(default_factory=time.time)   # filled in automatically
    level: Level = Level.INFO                      # default value
    source: str = ""
    text: str                                      # must be given
```

Options we use, and why:

- `kw_only=True`: arguments must be named, `LogLine(text="...", level=...)`. With four fields of similar types, it makes it impossible

to swap the code and the message by mistake.

- `slots=True` (only on `Sample`): the object uses less memory and is faster to create. It matters for `Sample`, made 500 times per second.
- `frozen=True` (only on `Param`): the object cannot be changed after creation. A safety bound must never be modified while the program runs.
- `field(default_factory=time.time)`: the time is taken *when each object is created*, not once when the program starts.

## 5.4 Enum: a closed list of allowed values

`Level` is an **enum**: a level can only be `DEBUG`, `INFO`, `WARNING`, `ERROR` or `FATAL`. Writing `Level.WARNIGN` (typo) fails at once instead of silently creating a sixth level. We use `StrEnum`, whose values are also texts, so they are saved to JSON as `"WARNING"` without conversion.

## 5.5 Abstract class: a contract

`Connector` is an **abstract base class** (ABC). It lists methods without writing them. A class that "inherits" from it (`class URSimConnector(Connector)`) must write every one of them, or Python refuses to create the object. It is a contract checked by the language.

## 5.6 Protocol: a contract without inheritance

`LLM` is a **Protocol**: it says "anything with a `create(system, messages, tools)` method". The real `ClaudeClient` and the tests' `ScriptedLLM` both qualify, without being related. We use it where the test double has nothing in common with the real thing.

## 5.7 TypedDict: a dictionary with known keys

LangGraph passes its state as a plain dictionary. `State` is a **TypedDict**: it lists the allowed keys and their types (`"report": dict | None`...), so a typo in a key is caught.

## 5.8 Generator: a function that produces items one by one

`signals()` never "returns" a list: it `yield`s one `Sample` at a time, forever, as packets arrive. The reader writes `for sample in connector.signals():` and gets each sample the moment it exists. This is what `Iterator[Sample]` means.

### 5.9 Threads, queues, locks, events

- A **thread** is a piece of the program running at the same time as the rest. The `Recorder` has three: one waits for signals, one for log lines, one for fault codes. Without them, waiting for a log line would block the signals.
- A **Queue** is a thread-safe mailbox: one thread puts events in, another takes them out. The loop mode reads alerts through one.
- A **Lock** makes sure two threads never touch the buffers at the same moment (one writing a sample while the other cuts a recording).
- An **Event** is a shared on/off flag: `Recorder.stop()` sets it, the threads see it and end.

### 5.10 deque: a list that forgets its oldest items

`deque(maxlen=5000)` keeps at most 5 000 items; adding the 5 001st silently drops the first. The recorder keeps the last 5 000 log lines and 1 000 fault codes this way, so memory never grows.

### 5.11 numpy array and NaN

A numpy array is a block of numbers stored side by side in memory, like a spreadsheet without formatting. It is about six times smaller than the same numbers as Python objects (measured below) and computations on whole columns are fast. **NaN** ("not a number") is the value of an empty cell: a channel missing from a sample.

## 6 The shared objects: `core/events.py`

**`events.py`** 123 lines

The shared vocabulary. Every other file speaks in these eight classes. They contain no logic, only data, so they are all dataclasses or enums.

**`Sample`** dataclass (slots, keyword-only) · core/events.py

**What** One instant of the signal stream: every channel measured at the same moment.

| Field | Type | Meaning, example |
|---|---|---|
| `t` | `float` | reception time, in seconds since 1 January 1970 (“Unix time”). `1793226730.07` is 28 Oct 2026, 23:32:10.07. |
| `values` | `dict[str, float]` | channel name → value. 63 channels on URSim, 24 on the fake arm. |

**Real example** A `Sample` received from URSim two seconds before the protective stop (12 of its 63 values):

```
Sample(t=1793226730.0686, values={
  "joint_1_q_actual": -1.7271,      "joint_1_q_target": -1.7271,  # rad
  "joint_1_current_actual": 0.6727, "joint_1_torque_target": 7.3079,  # A, Nm
  "joint_1_motor_temperature": 25.0,  # deg C
  "joint_4_current_actual": 0.0017,   # the wrist carries almost nothing
  "robot_mode": 7.0,      # 7 = RUNNING
  "safety_mode": 1.0,     # 1 = normal
  "speed_scaling": 0.0,   # no program running
  "main_voltage": 48.0, "tool_acc_z": -9.8139, ...})   # gravity
```

**Why** Grouping the channels by instant keeps them aligned in time: the shoulder current and the shoulder target torque of one `Sample` were measured together, so they can be compared without any resampling. A dictionary (rather than fixed fields) lets each machine have its own channels.

**Who** Created by `parse_realtime` (URSim) or `FakeConnector.signals()`, 500 per second. Read by `Recorder`, which copies the values into its numpy buffer and throws the object away.

### `Level` StrEnum · core/events.py

**What** The severity of a log line or fault code: `DEBUG < INFO < WARNING < ERROR < FATAL`.

**Why** Each source writes severity its own way; the connector translates them all to this one scale, so the rest of the code compares levels without knowing URSim:

| Source | Written as → Level |
|---|---|
| Controller log | `WARNING`, `ERROR`… → the same |
| PolyScope log | `WARN` → `WARNING` (with colour codes removed) |
| Primary port | a number 0–5 → `DEBUG`, `INFO`, `WARNING`, `WARNING`, `ERROR`, `FATAL` |
| Pendant log | no documented level: `INFO` for code `C0A0`, otherwise `WARNING` |

**Who** `get_recent_logs(min_level="WARNING")` keeps lines at that level or above; the `EventWatcher` raises an alert on `ERROR` and `FATAL`.

### `LogLine` dataclass · core/events.py

**What** One line of the machine's log, already understood.

| Field | Type | Meaning, example |
|---|---|---|
| `t` | `float` | reception time (Unix seconds). |
| `level` | `Level` | e.g. `Level.ERROR`. |
| `source` | `str` | which program wrote it: `"controller"`, `"polyscope"` or `"pendant"`. |
| `text` | `str` | the message alone, without date, level or colour codes. |

**Real examples** Raw line written by URSim → `LogLine` produced by `parse_log_line`:

```
RAW  2026-10-28_23:31:46:06655 00:11:42:676 ERROR   - RuntimeException: Syntax error
     on line 1: movej(get_actual_joint_positions(), a=1.0, v=)
OBJ  level=ERROR  source=controller
     text="RuntimeException: Syntax error on line 1: movej(get_actual_..., v=)"

RAW  <ESC>[1;31m2026-10-28 23:34:51.762 ERROR<ESC>[m -  ControllerSocket: 3 consecutive
     read timeouts, disconnecting <ESC>[1;31m{thread: ControllerConnectionThread, ...}
OBJ  level=ERROR  source=polyscope
     text="ControllerSocket: 3 consecutive read timeouts, disconnecting"

RAW  3.5 :: 0000d00h15m28.158s :: 2026-10-28 23:32:12.092 :: -2 :: C209A0:6 :: null
     :: 2 ::  ::  :: 0
OBJ  level=WARNING  source=pendant  text="C209A0"
```

**Why** Three files, three formats, colour codes, invisible characters: the agent should never see that mess. It sees one clean shape, whatever the machine.

**Who** Created by `URSimConnector.logs()` (live) and `read_flight_report` (black boxes); stored by `Recorder`; judged by `EventWatcher`; shown to Claude by `get_recent_logs` and after each test.

### `FaultCode` dataclass · core/events.py

**What** An official error raised by the controller.

| Field | Type | Meaning, example |
|---|---|---|
| `t` | `float` | reception time. |
| `code` | `str` | written as on the robot's tablet: C*code*A*argument*. `C209A0` = code 209, argument 0. |
| `message` | `str` | the source and text sent with it, e.g. `"[UNDEFINED]"`. |
| `level` | `Level` | `ERROR` by default; URSim sends `C209A0` at `WARNING`. |

**Real example** The only official fault code seen in the whole catalogue (5 runs out of 5), received 24 ms after `protective_stop()`:

```
FaultCode(t=+0.024 s, code="C209A0", message="[UNDEFINED]", level=WARNING)
```

**Why separate from `LogLine`** A fault code is an official, numbered, documented event: it always deserves an alert, and its number is the best search key in the documentation (`C209` → "A protective stop was triggered"). A log line is free text.

**Who** Created by `parse_primary`; stored by `Recorder`; always turned into an `Alert` by `EventWatcher`; shown by `get_recent_faults`.

### `Severity` StrEnum · core/events.py

**What** How serious an `Alert` is: `warning` or `critical`. Different from `Level`: `Level` is what the *machine* said, `Severity` is what *our rules* conclude. A `WARNING` log line containing "protective stop" gives a `critical` alert.

### `Alert` dataclass · core/events.py

**What** An anomaly found by the rules, that wakes the agent up in loop mode.

| Field | Type | Meaning, example |
|---|---|---|
| `t` | `float` | time of the event that caused it. |
| `reason` | `str` | one line for the operator, also the key for the 30 s repeat filter. |
| `severity` | `Severity` | `warning` or `critical`. |
| `evidence` | `list[str]` | the exact lines or codes that triggered it, given to Claude as is. |

**Real example** From the protective stop capture:

```
Alert(severity=critical, reason="log: Protective stop. Error code: C209A0",
      evidence=["Protective stop. Error code: C209A0"])
```

**Who** Created by `EventWatcher.feed`; shown in red by `CLI.alert`; written at the top of Claude's first message: "*Anomaly detected: log: Protective stop… Evidence: …*".

### `Param` dataclass (frozen) · core/events.py

**What** One numeric argument of a robot test, with its allowed range.

| Field | Type | Meaning, example |
|---|---|---|
| `kind` | `str` | JSON type: `"number"` or `"integer"`. |
| `low` | `float` | smallest allowed value, e.g. `-5`. |
| `high` | `float` | largest allowed value, e.g. `5`. |
| `description` | `str` | unit and meaning, read by Claude: `"amplitude in degrees"`. |

**ToolSpec** dataclass · core/events.py

**What** The description of one action a connector offers to Claude.

| Field | Type | Meaning, example |
|---|---|---|
| name | str | "joint_test". |
| description | str | what it does, for Claude. |
| params | dict[str, Param] | {"joint": Param(...), "delta_deg": Param(...)}. |
| moves_machine | bool | True: goes through operator approval and is recorded. |

Its method input_schema() turns it into the format the Claude API expects. Real output for joint_test:

```
{"type": "object",
 "properties": {
   "joint":     {"type": "integer", "minimum": 0,  "maximum": 5,
                 "description": "0 base, 1 shoulder, 2 elbow, 3-5 wrists"},
   "delta_deg": {"type": "number",  "minimum": -5, "maximum": 5,
                 "description": "amplitude in degrees"}},
 "required": ["joint", "delta_deg"], "additionalProperties": false}
```

**Why Param and ToolSpec?** The range "−5 to 5 degrees" is written *once*. The same object is shown to Claude (so it rarely asks for 30°) *and* enforced by SafetyPolicy (so it can never get 30°). Two separate lists would one day disagree.

# 7 The contract: core/interfaces.py

**interfaces.py** 69 lines

**Connector** abstract base class · core/interfaces.py

**What** What every machine must offer. There are no fields, only methods:

| Method | Returns | On URSim |
|---|---|---|
| signals() | Iterator[Sample] | reads port 30003, 500 samples/s |
| logs() | Iterator[LogLine] | tail -F on the three log files |
| faults() | Iterator[FaultCode] | reads port 30001 |
| docs() | list[str] | the files in manuals/ursim/ |
| tools() | list[ToolSpec] | the six tests |
| call_tool(name, args) | str \| Recording | runs a test, or opens a black box |
| stop() | None | sends stopj(2.0) |
| health() | list[tuple[bool, str]] | [(True, "dashboard: robot mode RUNNING"), ...] |
| snapshot() | dict[str, Any] | mode, safety, program, version (optional) |

**Why** The agent never imports `URSimConnector`: it only calls these methods. The tests pass a `FakeConnector` instead, and later the RoArm connector will slot in the same way.

**Who** Implemented by `URSimConnector` and `FakeConnector`; used by `Recorder` (the three streams), `Toolbox` (tools, docs, state) and `health`.

# 8 The memory: `core/recorder.py`

**`recorder.py`** 305 lines

**`Recording`** dataclass · core/recorder.py

**What** Everything that happened between two instants: signals, log lines and fault codes.

| Field | Type | Meaning, example |
|---|---|---|
| `id` | `str` | `"20261028-233227-f787"`: date, time and 4 random characters. |
| `label` | `str` | human description: `"joint 1 (shoulder) +3.0°"`. |
| `machine` | `str` | `"ursim_ur3"`. |
| `t0, t1` | `float` | start and end of the window (Unix seconds). |
| `t` | `np.ndarray` | the time of each sample, one row per sample. |
| `channels` | `dict[str, np.ndarray]` | one array per channel, same length as `t`; NaN where missing. |
| `logs` | `list[LogLine]` | the log lines inside the window. |
| `faults` | `list[FaultCode]` | the fault codes inside the window. |

**Real example** The capture of the protective stop:

```
Recording(id="20261028-233227-f787", machine="ursim_ur3",
          t0=1793226730.067, t1=1793226747.068)   # 17.0 s
len(rec.t)          -> 8187 samples
len(rec.channels)   -> 63
rec.duration        -> 17.0     (property: t1 - t0)
rec.rate_hz         -> 481.6    (property: samples per second)
rec.channels["safety_mode"] -> array([1., 1., ..., 3., 3.])
```

Methods:

- `resample(hz, channels)`: averages groups of samples to get fewer points. `rec.resample(10, ["safety_mode"])` gives 170 points instead of 8187. Averaging, rather than keeping one sample in 50, keeps the trace of a short peak.
- `save(folder)` writes the recording to a file named after its id, ending in `.json.gz`. `Recording.load(path)` reads it back. Compressed JSON: read-

able by any tool, no extra library.

**Why one class for all sources** A test, a black box and "the last 30 seconds" all become the same object, so the agent reads them all the same way (summary, plot, table).

**Who** Made by `Recorder.window/record` and `read_flight_report`; consumed by `Toolbox.describe_test`, `stream_gaps` and the functions of `context.py`.

`Recorder` class · core/recorder.py

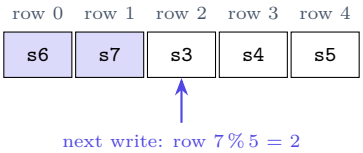
**What** Reads the three streams non-stop in the background and keeps the last two minutes, so any time slice can be cut out later.

| Field | Type | Meaning, example |
|---|---|---|
| `connector` | `Connector` | the machine read. |
| `directory` | `Path` | where recordings are saved: `data/recordings/`. |
| `_capacity` | `int` | rows of the ring buffer: 120 s × 500 Hz = 60 000. |
| `_t` | `np.ndarray` | 60 000 times, one per row. |
| `_data` | `np.ndarray` | 60 000 rows × one column per channel (63 on URSim). |
| `_columns` | `dict[str, int]` | channel name → column number: `{"joint_0_current_actual": 0, ...}`. |
| `_count` | `int` | samples received since the start (never reset). |
| `_logs` | `deque[LogLine]` | last 5 000 log lines. |
| `_faults` | `deque[FaultCode]` | last 1 000 fault codes. |
| `_subscribers` | `list[Queue]` | mailboxes that get every new event (loop mode). |
| `_lock` | `Lock` | one thread at a time in the buffers. |
| `_threads` | `list[Thread]` | the three readers. |
| `_stopping` | `Event` | raised by `stop()`. |
| `errors` | `dict[str, str]` | broken streams and why: `{"signals": "ConnectionError: ..."}`. |

(A name starting with `_` means "internal: only this class touches it".)

### The ring buffer, explained with 5 rows

Imagine a buffer of only 5 rows, and 7 samples `s1`…`s7` arriving. Each sample goes into row `count % 5` (the remainder of the division by 5), then `count` grows by one:




- `s1`...`s5` arrived with `count` = 0...4 and filled rows 0–4. `s6` arrived with `count` = 5, so it went to row 5 % 5 = 0; `s7` to row 6 % 5 = 1. They overwrote `s1` and `s2`, the oldest. `count` is now 7.
- Nothing is ever moved or deleted: writing a sample is one assignment, whatever the size.
- To read in time order, `window()` starts at the oldest row, `count % 5` = 2, and wraps around: rows 2, 3, 4, 0, 1 → `s3 s4 s5 s6 s7`.

The real buffer works the same with 60 000 rows. When a sample brings a channel name never seen before, a new column full of NaN is added on the right and its number is stored in `_columns`.

**Why a numpy ring buffer and not a Python list of `Samples`?** Measured on this laptop with 60 000 samples of 63 channels, the size of 120 s of URSim data: **30 MB** as one numpy array, **about 190 MB** as Python objects. A Raspberry Pi has 1 to 8 GB, shared with everything else. And a list would grow forever unless we deleted old items, which is slow; the ring overwrites them for free.

### The methods, and who calls them

- `start()`: starts the three reader threads. Each one loops: read the stream; if it breaks, note the cause in `errors`, wait 1 s, reconnect. Called once by `build_runtime`.
- `window(t0, t1)` → `Recording`: copies the rows whose time is between `t0` and `t1`, plus the log lines and fault codes of that period. Channels that stayed empty are left out.
- `latest(seconds)`: `window(now - seconds, now)`. Used by `get_recent_logs`, `get_recent_faults` and `health`.
- `record(action, label)`: notes the time minus 1 s, runs the test, waits 1.5 s, cuts the window and saves it. Because the buffer was already filling, the second *before* the test is there too. Used by `Toolbox` for every test.
- `subscribe()` → `Queue`: a mailbox that receives every new log line and fault code. Used by the loop mode and the benchmarks.
- `stop()`: raises `_stopping`; each thread ends at its next item.

**Why `stop()` exists.** During the fault captures, each run created a recorder and never stopped it. From round 4 on, the old recorders' connections were still open, and they added cuts and warnings to unrelated faults. `stop()` fixed it, a test checks it, and all 65 captures were redone.

# 9 The rules without AI: `core/checks.py`

**`checks.py`** 87 lines

`EventWatcher` class · core/checks.py

**What** Looks at each new log line or fault code, one at a time, and decides: alert or not.

| Field | Type | Meaning, example |
|---|---|---|
| `cooldown_s` | `float` | minimum time between two alerts with the same reason: 30 s. |
| `_patterns` | `list[re.Pattern]` | the 7 phrases below, compiled once (case ignored). |
| `_last` | `dict[str, float]` | reason → time of its last alert: `{"error in the log (polyscope)": 1793226737.1}`. |

**The rules** The one method, `feed(event)`, returns an `Alert` or `None`. In this order:

1. The event is a `FaultCode` → alert, reason `"fault code C..."`; `critical` if the code's level is `ERROR`/`FATAL`, otherwise `warning`.
2. The text contains one of the 7 **critical phrases** → `critical` alert, reason `"log: "` + the text: *protective stop, emergency stop, lost communication, connection lost/refused/reset, fault* (as a whole word: "default" does not match), *collision*, *overcurrent*.
3. The line's level is `ERROR` or `FATAL` → `warning` alert, reason `"error in the log (`*source*`)"`.
4. Anything else (INFO, DEBUG, WARNING without a critical phrase) → no alert.

Then the **repeat filter**: if the same reason gave an alert less than 30 s ago, the new one is dropped.

**Real runs** Each capture's events fed, in order, to a fresh `EventWatcher`:

| Fault | Events | Alerts produced |
|---|---|---|
| protective stop | 23 | *warning* fault code C209A0<br>*critical* log: Protective stop. Error code: C209A0<br>*warning* error in the log (polyscope): File attachment not found |
| invalid program | 7 | *warning* error in the log (controller): RuntimeException: Syntax error... |
| disk full | 33 | *warning* fault code C209A0 (the probe)<br>*warning* error in the log (polyscope): Could not write to FlightRecord zip file |
| network cut | 62 | *warning* error in the log (polyscope): ControllerSocket: 3 consecutive read timeouts |

What the examples show:

- **The repeat filter at work.** The invalid program wrote 2 ERROR lines, the disk full wrote 20 (a Java error and its stack trace, one line each): each gave *one* alert, because they share the reason `"error in the log (...)"`.
- **A WARNING is not enough.** Before disconnecting, PolyScope wrote `WARNING Controller monitoring timed out too many times`. No critical phrase, not an ERROR: no alert. The alert came 10 s later with the ERROR line.
- **The fault code comes as a warning**, because URSim sends `C209A0` at level `WARNING`; the *log line* about the same event is what makes it critical.

**Who** Created by `build_graph`; fed by the `watch` step with each event from `Recorder.subscribe()`.

### The function `stream_gaps(recording, max_gap_s=0.1)`

Not a class: a function returning `list[tuple[float, float]]`, the holes longer than 0.1 s in the signal stream, each as `(start, duration)` in seconds from the window start.

- How: it puts the window start before the sample times and the window end after them, computes the differences between neighbours, and keeps those above 0.1 s. At 500 Hz the normal difference is 0.002 s.
- Why the window edges: if the stream stops for good, there is no "next sample" to make a difference with; the end of the window plays that role.
- Real values from the captures: network cut `[(0.012, 15.206)]`, the stream stopped 0.012 s after the fault and never came back; controller frozen `[(0.013, 9.01)]`, a 9 s hole then the stream resumed; CPU starvation `[(3.219, 0.1), (5.919, 0.1)]`, many short holes.

Used by `Toolbox.describe_test` (“Signal stream cuts: . . . ”) and by the capture tool.

# 10 Safety: `core/safety.py`

**`safety.py`** 76 lines

**`SafetyRefusal`** exception class · core/safety.py

**What** The error raised when an action is refused. Its message goes back to Claude word for word, so Claude knows why and can change plan.

**`SafetyPolicy`** dataclass · core/safety.py

**What** Decides whether a robot action may run.

| Field | Type | Meaning, example |
|---|---|---|
| `ask_operator` | `Callable[[str], bool]` | asks a yes/no question; in practice `CLI.confirm`. |
| `pin` | `str` | the operator PIN, `""` for none. Never logged. |
| `ask_pin` | `Callable[[], str] \| None` | asks for the PIN; in practice `CLI.ask_pin`. |
| `_pin_given` | `bool` | the PIN was already given in this investigation. |

**Real outputs** `check(spec, args)` on `joint_test`:

```
args {"joint": 1, "delta_deg": 30}
  -> REFUSED: joint_test: delta_deg=30.0 outside the allowed range [-5, 5]
args {"joint": 1}
  -> REFUSED: joint_test: argument delta_deg is required
args {"joint": 1, "delta_deg": 3}
  -> asks: "The agent wants to run joint_test(joint=1, delta_deg=3). Allow?"
     operator says no -> REFUSED: the operator refused this test; try another
                         lead or conclude with the data you have
```

**Why functions as fields (`Callable`)?** The policy does not know how to ask: in the terminal it is `input()`, in the tests it is `SilentHMI.confirm` that answers “yes” or “no” instantly, later it could be a web button. Passing the function in keeps `SafetyPolicy` identical in all cases.
**Why refuse rather than clamp?** If Claude asks for 30° and silently gets 5°, its reasoning is based on a test it did not get. A refusal is honest.

**Who** Created by `build_runtime`; `reset()` is called at the start of each investigation; `check()` is called by `Toolbox.execute` before every machine action.

# 11 The URSim connector: connectors/ursim.py

**ursim.py** 672 lines

The biggest file, on purpose: the only place that knows the Universal Robots formats (your decision: one file). Two classes and six functions.

## 11.1 The robot's doors (network ports)

| Port | Name | What we do with it |
|---|---|---|
| 30003 | Real-Time | Read the signals: one binary packet every 2 ms. |
| 30001 | Primary | Read the fault codes. |
| 30002 | Secondary | Send URScript programs for the tests. |
| 29999 | Dashboard | Text commands: mode, safety status, power on, unlock. |

## 11.2 The functions

**parse_realtime(packet: bytes, t: float) -> Sample**. A Real-Time packet is 1 000+ bytes of numbers with no names. The table REALTIME_FIELDS (from the official "Remote Control via TCP/IP" document) says where each group sits. One entry:

```
"q_actual": (252, 6, ["joint_0_q_actual", ..., "joint_5_q_actual"]),
# bytes 252 to 299: 6 numbers of 8 bytes (big-endian doubles), one per joint
```

16 entries give the 63 channels. A field beyond the end of a shorter packet (older controller) is skipped instead of crashing.

**parse_primary(packet, t) -> FaultCode | None**. Most Primary packets are robot state; only "Robot Message" packets (type 20) can be faults. It reads the code, argument and level numbers and builds C209A0. Returns None for everything else.

**parse_log_line(line, previous) -> LogLine | None**. Shown with real examples in the LogLine section. It recognises the file from the date format (2026-10-28_23:... = controller, 2026-10-28 23:... = PolyScope, 3.5 :: = pendant). A line in no format (a Java stack trace line) continues the previous message and inherits its level and source: that is why previous is passed in. Connections of clients (ours included) are demoted to DEBUG so they never raise an alert.

**read_flight_report(path, machine) -> Recording**. Opens a black box zip (metadata.json, summary.log, files/realtimedata.csv with 15 000 rows × 181 columns, logs). Column names are renamed to the live names ("Actual

`Current [A] J1"` → `joint_1_current_actual`, `"Target Current [A] J1"` → `joint_1_current_target`) so Claude compares like with like. The time column is the controller's uptime; the last row is aligned on the "Reporting Time" of `summary.log`.

**`send_script(host, script)`**. Sends URScript to port 30002. Several lines are wrapped in `def robot_agentic(): ... end`, the only form the controller runs as a program.

**`line_time(line) -> float | None`**. The date at the start of a log line, as Unix seconds. Used for black boxes and by the capture tool.

### `Dashboard` class · connectors/ursim.py

**What** A small client for port 29999, where you type text commands and get text answers.

| Field | Type | Meaning, example |
|---|---|---|
| `host` | `str` | `"127.0.0.1"`. |
| `timeout` | `float` | 5 s to connect and answer. |

**Example**

```
dashboard.send("robotmode")      -> "Robotmode: RUNNING"
dashboard.robot_mode()           -> "RUNNING"
dashboard.safety_status()        -> "PROTECTIVE_STOP"   (after the fault)
dashboard.power_on_and_release() -> "RUNNING"   (closes popups, unlocks,
                                                 powers on, releases brakes)
```

**Why a class** It hides the connection details and the answer prefixes. Each command opens its own connection: a few ms more, but nothing to repair after a controller restart, which happens all the time during fault injection.

**Who** `URSimConnector` (end of tests, state, recovery) and `URSimFaults` (repairs).

### `URSimConnector` class, implements Connector · connectors/ursim.py

**What** The UR3e robot simulated by URSim, as the rest of the code sees it.

| Field | Type | Meaning, example |
|---|---|---|
| `name` | `str` | `"ursim_ur3"`, written in reports and recordings. |
| `host` | `str` | controller address, `"127.0.0.1"`. |
| `container` | `str` | Docker container name, `"ursim"`, to read the logs. |
| `manuals_dir` | `Path` | `manuals/ursim/`. |
| `dashboard` | `Dashboard` | its Dashboard client. |
| `_payload_offset_kg` | `float` | extra payload left declared by an interrupted `add_load`; normally 0. |

**The six tests** (`tools()` returns them as `ToolSpecs`)

| | |
|---|---|
| `joint_test` | one joint ±5° max at 10 % speed, and back. |
| `move_hand` | the hand ±2 cm max in a straight line, and back. |
| `add_load` | +0 to 1.5 kg declared payload, shoulder test, original payload restored. |
| `overall_test` | each joint in turn, 1–5°. |
| `recover_robot` | close popups, unlock, power on, release brakes. |
| `get_flight_report` | open black box number `index` (0 = newest). Moves nothing. |

**Example: `_payload_offset_kg`** Claude runs `add_load(kg=1)`. The program declares +1 kg, but a protective stop interrupts it before the line that restores the payload. The connector notes `_payload_offset_kg = 1.0` and tells Claude "*the extra 1 kg payload is still declared, recover_robot will remove it*". When Claude calls `recover_robot`, the connector subtracts the 1 kg. Without this field, our own test would leave a wrong payload: the very fault we look for.

**Who** Created by `build_runtime` when `CONNECTOR=ursim`.

**A trap found on the real simulator.** Stopping our `docker exec` did not stop the `tail` it had started inside the container. Now `tail` runs in the background of a small shell that waits on our input; when we stop, the input closes and the shell kills `tail`.

`FakeConnector` class, implements Connector · connectors/fake.py

`fake.py` 136 lines

**What** A fake six-joint arm in pure Python, for the tests.

| Field | Type | Meaning, example |
|---|---|---|
| `q` | `list[float]` | joint positions (rad), starts at `[0, -1.57, 1.57, -1.57, -1.57, 0]`. |
| `qd` | `list[float]` | joint speeds (rad/s). |
| `friction` | `list[float]` | effort factor per joint, 1.0 = healthy. |
| `log_q, fault_q` | `Queue` | lines and codes waiting to be read by the streams. |
| `calls` | `list[tuple[str, dict]]` | every action received, to check in the tests. |

**Example (a real test)** `fake.inject("friction", joint=2, factor=1.6)` raises joint 2's effort and logs `WARNING Joint 2: high following error`. The fake Claude asks `joint_test(joint=2, delta_deg=5)`; the test checks that Claude received a text, an image and a table with `joint_2_current_actual` and more than 50 rows.

**Why** URSim takes a minute to start and needs Docker. The fake arm runs the 24 tests in seconds, anywhere, with the same contract.

## 12 The agent: `src/robot_agentic/agent/`

**`State`** TypedDict · agent/graph.py

**`graph.py`** 251 lines

**What** What the steps of the graph pass to each other.

| Field | Type | Meaning, example |
|---|---|---|
| `mode` | `str` | `"loop"` or `"check"`. |
| `alert` | `Alert \| None` | the alert that woke the agent (None in check mode). |
| `approved` | `bool` | the operator's answer. |
| `request` | `str \| None` | the operator's question in check mode. |
| `report` | `dict \| None` | the report from `submit_report`. |
| `report_path` | `str` | where it was saved. |
| `turns` | `int` | number of calls to Claude. |
| `error` | `str \| None` | why it ended without a report, e.g. `"maximum number of turns reached"`. |

**The graph** `build_graph` creates four steps and the arrows between them:

$$\texttt{watch} \xrightarrow{\text{alert}} \texttt{ask_approval} \xrightarrow{\text{yes}} \texttt{investigate} \rightarrow \texttt{report}$$

Each step is a function that receives the `State` and returns the keys it changes, e.g. `ask_approval` returns `{"approved": True}`. `investigate` is the conversation with Claude: send, run the requested tools, add their results, send again, until

`submit_report` or 25 turns. A failing tool returns its error to Claude instead of crashing.

> **The conversation is only ever extended, never edited.** This is what makes prompt caching work (each turn pays full price only for its new part) and keeps Claude's reasoning between turns.

`run_check` and `run_loop` are the two modes; `run_loop` repeats the graph forever and drops the events received during an investigation (mostly caused by the agent's own tests).

### `Toolbox` class · agent/tools.py

**`tools.py`** 239 lines

**What** The list of tools Claude can call, and the code that runs them.

| Field | Type | Meaning, example |
|---|---|---|
| `connector` | `Connector` | the machine. |
| `recorder` | `Recorder` | the memory. |
| `safety` | `SafetyPolicy` | the gate. |
| `budget` | `int` | tokens allowed for one table of values: 40 000. |
| `machine_tools` | `dict[str, ToolSpec]` | the connector's tests by name. |
| `docs` | `DocSearch \| None` | the documentation index (None if no document). |
| `report` | `dict \| None` | filled by `submit_report`; the graph watches it to stop. |
| `used_recordings` | `list[str]` | ids of the recordings made, saved in the report. |

**Its tools** Common ones: `get_status`, `get_recent_logs(seconds, min_level)`, `get_recent_faults(seconds)`, `search_docs(query)`, `get_recording(recording_id, channels, hz)` (`"latest:30"` = the last 30 s), `plot_recording`, `submit_report`; plus the connector's six.

**Example: one test** What happens when Claude asks for a test, i.e. `execute("joint_test", {"joint": 1, "delta_deg": 3})`:

1. `joint_test` is a machine tool → `safety.check` (bounds, operator, PIN).
2. It moves the machine → `recorder.record(...)` runs it and returns a `Recording`.
3. The id is added to `used_recordings`.
4. `describe_test` builds the answer: three blocks, (1) a text in this format: "*Recording <id> (<duration> s, <points> points at <rate> Hz): joint 1 (shoulder) +3.0°: finished; mode <mode>, safety <status>*", followed by any

cuts, fault codes and log warnings; (2) a PNG plot of the 6 channels that moved most; (3) the table of every channel that varied.

`submit_report` has a strict schema: `root_cause`, `confidence` (low, medium or high), `evidence`, `repair`, `references`, `tests_done`. Claude cannot hand in a report with a field missing.

**Why** One object gathers everything a tool needs, so the graph just says `toolbox.execute(name, args)` and never cares which tool it is.

## The functions of `context.py`

**`context.py`** 113 lines

How a `Recording` is shown to Claude within the token budget.

- `moving_channels(rec) -> list[str]`: the channels that vary, those that vary most first; constant ones are dropped. On the protective stop recording (robot at rest), only 2 of 63 channels varied: `["speed_scaling", "safety_mode"]`. That is exactly the story of the fault.
- `csv_within_budget(rec, channels, budget) -> (text, rate)`: starts at full rate and halves it until the table fits. Real result on the same recording:

```
rate 481.6 Hz, about 32,799 tokens (fits in 40,000)
t_s,speed_scaling,safety_mode
0.001,0,1
0.003,0,1
```

- `plot(rec, channels)`: up to 8 channels, one per row, as a PNG in memory.

**Why a plot *and* a table?** Research (Zhou & Yu, ICLR 2025) shows LLMs spot anomalies in time series better on an image than in a list of numbers. The table keeps the exact values. You chose to give both.

**`LLM`** Protocol · agent/llm.py

**`llm.py`** 141 lines

**What** "Anything with a `create(system, messages, tools)` method that returns an object with `content` and `stop_reason`." The graph only knows this.

**Why** The tests pass a `ScriptedLLM` that replays prepared answers: no network, no cost, same result every time.

**`Usage`** dataclass · agent/llm.py

**What** Running totals of one client's calls.

| Field | Type | Meaning, example |
|---|---|---|
| `calls` | `int` | number of calls. |
| `input_tokens` | `int` | tokens sent at full price (including cache writes). |
| `output_tokens` | `int` | tokens written by Claude (thinking included). |
| `cache_read_tokens` | `int` | tokens re-read from the cache, 20 times cheaper. |
| `cost_usd` | `float` | estimated dollars. |

Printed at the end of every command, in this format: "*Claude calls: <n>, <n> input tokens, <n> output, ≈$<cost>*".

**`ClaudeClient`** class, satisfies LLM · agent/llm.py

| Field | Type | Meaning, example |
|---|---|---|
| `client` | `anthropic.Anthropic` | the official SDK client. |
| `model` | `str` | `"claude-opus-5-5"`. |
| `effort` | `str` | `"high"` (your choice). |
| `log_path` | `Path \| None` | `data/llm_calls.jsonl`. |
| `usage` | `Usage` | the totals. |

**What `create` sends** adaptive thinking (Claude decides how much to think, a summary is shown), effort high, a cache point on the system prompt plus automatic caching of the conversation, and a server fallback model if the main one declines. After each call it adds one line to `llm_calls.jsonl`: tokens, cost, duration.

**Cost example** `cost_usd` with the Opus 5.5 prices ($4 in, $20 out, $0.20 cache read per million tokens): a turn with 2 000 new input tokens, 30 000 read from cache and 1 500 output costs $2000 \times 4 + 30000 \times 0.2 + 1500 \times 20 = 44\,000$ millionths of a dollar, **$0.044**. Without the cache, the 30 000 tokens alone would cost $0.12.

**`prompts/system.md`** 27 lines

The agent's instructions, in plain English: what it has, the method (facts → ranked hypotheses → the smallest test that separates them → conclude), the rules.

# 13 Around the agent

`Passage` dataclass · docs.py

**`docs.py`** 150 lines

**What** An extract of a document, about 1 500 characters.

| Field | Type | Meaning, example |
|---|---|---|
| `source` | `str` | file path: `manuals/ursim/ur_error_codes_SW5_26.md`. |
| `page` | `int \| None` | page number for PDFs, None for text files. |
| `text` | `str` | the extract. |

`cite()` gives the reference to quote: `"ur_error_codes_SW5_26.md"` or `"service_manual.pdf, p. 12"`.

`DocSearch` class · docs.py

**What** Ranks passages for a question with BM25, the formula of classic search engines: a word scores high when it is rare in the whole documentation and frequent in a short passage.

| Field | Type | Meaning, example |
|---|---|---|
| `passages` | `list[Passage]` | 280 passages today (225 error codes, some split). |
| `docs` | `list[Counter]` | the words of each passage, counted. |
| `lengths` | `list[int]` | words per passage. |
| `idf` | `dict[str, float]` | how rare each word is. |
| `k1, b` | `float` | the two standard BM25 settings (1.5, 0.75). |

**Real example** `search("C209A0 protective stop", k=2)`:

```
19.38  ## C209 A protective stop was triggered (for test purposes only)
       Source: https://www.universal-robots.com/manuals/EN/HTML/SW5_26/
               Content/prod-err-codes/topics/CODE_209.html
10.65  ## C162 The protective stop was likely caused by incorrectly specified
       payload mass and/or center of gravity.
```

`tokenize("Fault C204A3 on joint")` gives `["fault", "c204a3", "on", "joint", "c204"]`: the short form `c204` is added so the code matches its page title.

**Who** Built once by `Toolbox` from `connector.docs()` (cached in `data/docs_index/`); used by the `search_docs` tool.

**Settings** dataclass · config.py

**config.py** 82 lines

**What** The settings of one run. Each field is read from the environment variable of the same name in capitals, or from `.env`; otherwise the default applies.

| Variable | Type | Default |
|---|---|---|
| CONNECTOR | str | ursim (or fake) |
| ROBOT_HOST | str | 127.0.0.1 |
| URSIM_CONTAINER | str | ursim |
| CLAUDE_MODEL | str | claude-opus-5-5 |
| RECORDING_TOKEN_BUDGET | int | 40000 |
| OPERATOR_PIN | str | empty (no PIN) |
| LOG_LEVEL | str | WARNING |
| DATA_DIR | Path | data |
| MANUALS_DIR | Path | manuals |

Derived folders: `recordings_dir`, `reports_dir`, `llm_log`, `log_file`, all inside `data_dir`, so a test can point everything at a temporary folder with one setting.

**CLI** class · hmi.py

**hmi.py** 77 lines

**What** The terminal interface, the only code allowed to `print`. One field: `auto_approve: bool` (the `-yes` option, simulator only). Methods: `alert` (red), `confirm` (`[y/N]`, only "y"/"yes" mean yes), `ask_pin` (typed without being shown), `say`, `thinking` (grey), `tool_call` (blue), `checklist`, `report`.

**Why** Everything the operator sees goes through one object: a web interface later only has to offer the same methods.

**Runtime** dataclass · __main__.py

**__main__.py** 141 lines

**What** The assembled system: `settings: Settings`, `toolbox: Toolbox`, `hmi: CLI`. `build_runtime()` makes it: connector → recorder → safety → toolbox. The command line and the benchmarks both use it, so they run exactly the same system.

# 14 Outside the package: tools, benchmarks, tests

**URSimFaults** class · tools/fault_injection/inject.py

**inject.py** 233 lines

**What** Knows how to provoke and repair the 13 faults.

| Field | Type | Meaning, example |
|---|---|---|
| host | str | controller address. |
| container | str | "ursim". |
| freeze_s | float | how long the "freeze" fault lasts: 8 s. |

Methods: `inject(fault)`, `revert(fault)`, `wait_ready()` (back to RUNNING). Example: `inject("cpu_starvation")` runs `docker update -cpus 0.1 ursim`; `revert("cpu_starvation")` gives back all the CPUs.

**Who** The capture tool, the benchmarks, and you on the command line.

**capture.py** 207 lines

Functions only: `capture(fault, seconds, out_dir)` provokes one fault and saves everything (raw new log lines, fault codes, mode changes, cuts, black boxes, recovery time) in one JSON per run in `data/catalogue/`.

**catalogue.py + catalogue.yaml + template** 234 + 143 + 79 lines

Builds the fault catalogue PDF; nothing observed is written by hand.

**benchmarks/run.py** 148 lines

For each scenario of `scenarios.yaml`: repair, inject, run the agent, grade the root cause by keywords and by a Claude judge (`judge()` returns `(correct: bool, reason: str, cost: float)`), repair. Saves the results in `benchmarks/results/`.

## The test doubles: `tests/conftest.py`

**SilentHMI** dataclass · tests/conftest.py

Replaces `CLI`: `approve: bool` decides every answer, `log: list[str]` notes everything "shown". A test sets `approve=False` to check that a refused movement never reaches the robot.

`ScriptedLLM` class, satisfies LLM · tests/conftest.py

Replaces Claude: `turns` is the list of tool calls to make at each turn; `requests` keeps what the agent sent, so a test can check exactly what Claude would have seen.

### The 24 tests

`test_agent.py` (5): a check produces a report; every movement needs approval; an out-of-range test never moves; loop mode wakes on a fault and asks; declining stops it. `test_core.py` (10): safety, schema, watcher, ring buffer, recorder stop, save/load, cuts, settings. `test_ursim.py` (9): packets, log formats, pendant, black box, documentation search.

# 15 How it was developed

## 15.1 The code principles (CONTRIBUTING.md)

1. **Type annotations everywhere**.
2. **Docstrings everywhere** (your choice): about a quarter of the lines are explanations.
3. **Deep modules** (J. Ousterhout): a simple interface hiding a rich implementation; no tiny functions called once; each file hides one decision.
4. **Logging**: every module logs; secrets are never logged.
5. **Comments only for the why**.
6. **English everywhere**; `ruff` and `pytest` pass before every commit.

## 15.2 The history of this branch

| | |
|---|---|
| `3f6230e` | Refactor: software detection only, English, no baselines, redundancy removed. |
| `149501b` | URSim study: evidence capture, pendant log, black boxes, stream-cut fixes. |
| `22d0ba1` | Official UR error code pages as documentation. |
| `55d2ac9` | Four new faults. |
| `d375ff4` | Network and disk faults; URSim on its own disk. |
| `e27d060` | Capture probe for disk full, `-repeat`. |
| `934bbc3` | Catalogue generator. |
| `f48d821` | `Recorder.stop()`: fixes the polluted captures. |
| `f6c35d2` | The fault catalogue (13 × 5 runs) and detection ideas. |
| `cfd87a4` | The first version of this guide. |

### 15.3 Bugs found only by testing on the real simulator

- `tail` kept running inside the container after we stopped.
- A stream that stopped for good was not counted as a cut.
- Recorders were never stopped and polluted the captures (all 65 redone).
- Log files contain NUL characters and get rewritten: reading by line number failed.
- Killing PolyScope stops the whole container, so that fault became a freeze.
- `docker update -cpus 0` does not remove a CPU limit.
- The mean sample rate looked normal for a stream that had stopped.

## 16 How big is it?

| Part | Total | Code | Docstr. | Comm. | Blank |
|---|---|---|---|---|---|
| Agent package (`src/`) | 2 668 | 1 532 | 678 | 28 | 430 |
| Fault tools (`tools/`) | 759 | 492 | 137 | 19 | 111 |
| Benchmarks | 149 | 107 | 21 | 1 | 20 |
| Tests | 389 | 256 | 47 | 0 | 86 |
| **All Python** | **3 965** | **2 387** | **883** | **48** | **647** |

Plus about 820 lines of Markdown, YAML and LaTeX.

**In short:** about **4 000 lines of Python**, of which **about 2 400 are real code**. The agent itself is **2 668 lines in 19 files** (1 532 of real code), defining **27 classes** (plus 2 test doubles and `URSimFaults` outside it). On `main` it was 2 917 lines in 33 files.

| File | Total | Code | Classes |
|---|---|---|---|
| connectors/ursim.py | 672 | 426 | Dashboard, URSimConnector |
| core/recorder.py | 305 | 190 | Recording, Recorder |
| agent/graph.py | 251 | 145 | State |
| agent/tools.py | 239 | 164 | Toolbox |
| docs.py | 150 | 91 | Passage, DocSearch |
| ___main___.py | 141 | 84 | Runtime |
| agent/llm.py | 141 | 75 | LLM, Usage, ClaudeClient |
| connectors/fake.py | 136 | 72 | FakeConnector |
| core/events.py | 123 | 49 | 8 classes |
| agent/context.py | 113 | 48 | (functions) |
| core/checks.py | 87 | 36 | EventWatcher |
| config.py | 82 | 48 | Settings |
| hmi.py | 77 | 41 | CLI |
| core/safety.py | 76 | 38 | SafetyRefusal, SafetyPolicy |
| core/interfaces.py | 69 | 24 | Connector |

# 17 What is missing

## 17.1 1. The detection rules on the signals

`EventWatcher` only reads logs and fault codes. Rules b (target vs reality), e (joint effort ratio), a (hardware limits) and c (recent past) are decided but **not written**, and `config/detection.yaml` does not exist. In loop mode, **4 of the 13 faults raise no alert** (wrong payload, controller frozen, CPU starvation, CPU overload).

## 17.2 2. The expected current in the live stream

Rules b and e need the current the controller *expects*. Only black boxes have it (`current_target`); `REALTIME_FIELDS` reads the target torque but not the target current. Its position in the packet must be checked on URSim first.

## 17.3 3. A “stream health” rule (proposed rule g)

An alert when fewer samples arrive than expected, or none. It would catch 8 of the 13 faults, 3 of them missed by the logs. Waiting for your decision.

## 17.4 4. Benchmarks: never run, incomplete

- **The agent has not yet done a full investigation with the real Claude on this branch** (no report, no benchmark result, no `llm_calls.jsonl`). It

is tested with `ScriptedLLM` only.

- `scenarios.yaml` covers 5 of the 13 faults, so the statistics do not exist yet.

### 17.5 5. Smaller gaps, some found while writing this guide

- **Misleading rate.** `Recording.rate_hz` counts samples between the first and last one. In the network cut capture it says **481.3 Hz** although the stream stopped after 0.012 s for 15.2 s. Claude gets the cut list too, so nothing is hidden, but the rate in the summary should be the effective one (samples ÷ window length), as in the catalogue.
- **Continuation lines inherit ERROR.** In the network cut capture, a long PolyScope line about a licence certificate was split in two; the second half has no date, so `parse_log_line` gave it the level of the line above (ERROR). Here no alert resulted (same reason, repeat filter), but a harmless line could trigger one.
- **No "unsubscribe".** Each `Recorder.subscribe()` queue fills forever; the benchmarks create one per scenario. A slow memory leak over long series.
- **Documentation**: only the error code pages; no User or Service Manual, so no repair procedures or hardware limits (needed by rule a).
- **Tests**: none for the fault tools, the benchmark runner, `cost_usd`, the plot/table budget; no automatic test against URSim; no continuous integration.
- An old `data/baselines/` folder from before the refactor is still on disk (not in git).

### 17.6 6. Planned for later

The RoArm-M2-S connector (with our own log lines and codes), physical anomaly detection (ESP32 → USB → Raspberry Pi), running on the Pi, and an interface beyond the terminal.

## 18 Glossary

**URSim**
The official Universal Robots simulator: the real controller software, without the arm.

**URControl**
The controller program: computes movements, serves ports 30001–30004.

**PolyScope**
The robot's user interface (on the tablet). Serves the Dashboard (29999) and writes the black boxes.

**Teach pendant**
The tablet used to program and drive the robot. Its event list is `log_history.txt`.

**URScript**
The robot's programming language (`movej`, `set_target_payload`...).

**Protective stop**
The robot stops because a safety limit was crossed; code `C209` when triggered by a program.

**Robot mode / safety mode**
e.g. RUNNING (7) / NORMAL (1), PROTECTIVE_STOP (3).

**Speed scaling**
The fraction of the programmed speed the robot actually uses; 0 when nothing runs.

**Black box (flight report)**
Zip written by the robot on each error: about 30 s of its own data and logs.

**Payload**
The mass declared for the tool. A wrong value makes the expected torques wrong.

**Following error**
The gap between the position the controller asked for and the real one.

**Unix time**
Seconds since 1 January 1970; how computers store instants.

**Big-endian double**
A decimal number stored in 8 bytes, most significant byte first: the UR binary format.

**Ring buffer**
A fixed-size memory where new data overwrites the oldest.

**BM25**
The ranking formula of classic search engines.

**Token**
The unit Claude reads and is billed by, about 3–4 characters of text.

**Prompt caching**
The API keeps the start of a conversation, so resending it costs much less (20 times less for Opus 5.5).

**Adaptive thinking / effort**
Claude decides how long to think; "effort" sets how thorough it is.

**LangGraph**
A library to describe a program's flow as a graph of steps.

**netem / iptables**
Linux tools to delay or lose network packets / block a port.

**Docker container**
An isolated mini-system running one program (here URSim) on the laptop.